AI Security Findings

Overall risk: CRITICAL | Total findings: 12

Executive summary

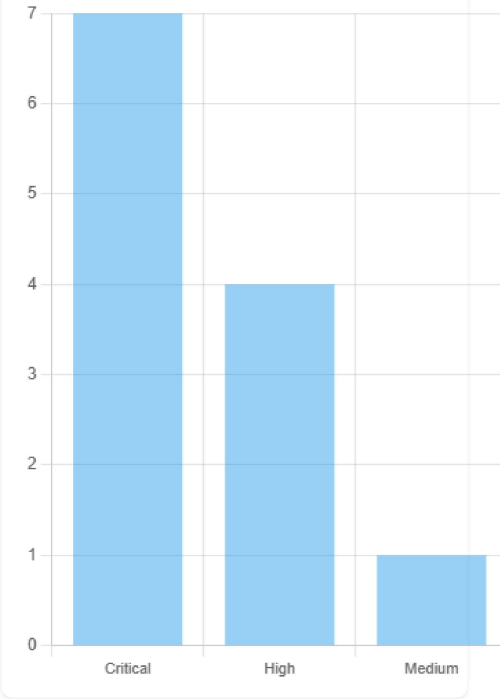
Risk overview

Total findings: 12. Critical: 7, High: 4, Medium: 1, Low: 0.

Technical summary

[
{
"title": "SQL Injection in store_training_sample()",
"severity": "critical",
"location": "Function store_training_sample (lines ~17-24)",
"description": "Untrusted input is directly concatenated into SQL queries, enabling SQL injection attacks that can compromise the database.",
"recommendation": "Use parameterized queries or prepared statements to safely insert user inputs into SQL commands."
},
{
"title": "Shell Injection in fine_tune_on_user_feedback()",
"severity": "critical",
"location": "Function fine_tune_on_user_feedback (lines ~26-34)",
"description": "Executing subprocess.run with shell=True on untrusted input allows arbitrary command execution.",
"recommendation": "Avoid shell=True; use argument arrays and sanitize or validate all inputs before subprocess execution."
},
{
"title": "Stored Cross-Site Scripting (XSS) via Unsanitized LLM Output",
"severity": "critical",
"location": "Function render_llm_response (lines ~40-52)",
"description": "LLM output is injected directly into HTML without sanitization, risking stored XSS attacks.",
"recommendation": "Sanitize or escape all LLM-generated content before embedding into HTML output."
},
{
"title": "Arbitrary Code Execution via exec() of LLM-generated Code",
"severity": "critical",
"location": "Function execute_llm_generated_code (lines ~54-65)",
"description": "LLM-generated Python code is executed directly without sandboxing, risking server compromise.",
"recommendation": "Avoid direct execution of LLM output; implement sandboxing, manual review, or strict code vetting."
},
{
"title": "Excessive Agency: Unrestricted LLM Access to Destructive Tools",
"severity": "critical",
"location": "Function autonomous_agent_action (lines ~81-101)",
"description": "The LLM is granted unrestricted autonomous operations (email, DB deletion, shell commands) without approval, enabling high-impact misuse.",
"recommendation": "Enforce strict access controls, human-in-the-loop confirmation, and restrict LLM capabilities."
}
]

Severity distribution



Severity	Title	File / Location	Description	Recommendation
CRITICAL	SQL Injection in store_training_sample()	`Function store_training_sample (lines ~17-24)`	The function builds an SQL query by concatenating untrusted user input directly into the query string. This is susceptible to SQL injection, allowing attackers to execute arbitrary SQL commands on the database.	Use parameterized queries or prepared statements to safely insert user input into SQL commands.
HIGH	Unvalidated Data Poisoning via Training Sample Storage	`Function store_training_sample (lines ~17-24)`	User input is stored unfiltered as training data without validation or sanitization. This allows an attacker to inject poisoned data influencing future model behavior maliciously.	Validate and sanitize training data inputs. Implement adversarial data detection mechanisms before ingestion.
CRITICAL	Shell Injection in fine_tune_on_user_feedback()	`Function fine_tune_on_user_feedback (lines ~26-34)`	The function invokes subprocess.run with shell=True using an untrusted filename as part of the command. This allows arbitrary command injection by crafting the feedback_file parameter.	Avoid shell=True or sanitize inputs properly. Use subprocess with argument lists and avoid passing untrusted data as raw shell commands.
CRITICAL	Stored Cross-Site Scripting (XSS) via Unsanitized LLM Output	`Function render_llm_response (lines ~40-52)`	LLM output is directly injected into HTML without HTML-encoding or sanitization. An attacker can prompt the LLM to generate malicious HTML/JavaScript payloads leading to stored XSS.	Escape or sanitize all LLM-generated content before embedding into HTML. Use libraries for safe HTML rendering.
CRITICAL	Arbitrary Code Execution via exec() of LLM-generated Code	`Function execute_llm_generated_code (lines ~54-65)`	LLM-generated Python code is executed directly via exec() without any sandboxing or manual review, allowing malicious code execution and server compromise.	Avoid executing LLM output directly. Implement sandboxing, manual approval workflows, or strict code verification before execution.
CRITICAL	Command Injection via os.system() of LLM Command	`Function run_llm_shell_command (lines ~67-79)`	Shell commands generated by the LLM are executed directly by os.system() without sanitization or allowlisting. Malicious commands can cause severe system damage.	Never execute untrusted shell commands directly. Use an allowlist for permissible commands or disallow shell execution altogether.

Severity	Title	File / Location	Description	Recommendation
CRITICAL	Excessive Agency: Unrestricted LLM Access to Destructive Tools	`Function autonomous_agent_action (lines ~81-101)`	The LLM agent is granted autonomous access to destructive operations like email sending, database deletion, shell execution, and fund transfer with no human approval or scope limitations.	Enforce strict access controls, human-in-the-loop confirmation, and scope restrictions on agent actions to prevent misuse.
HIGH	High-risk Decisions Fully Delegated to LLM Without Human Review	`Function auto_approve_llm_decision (lines ~103-115)`	The LLM makes binary approval decisions (e.g., loans, access grants) with no human oversight, auditing, or appeals process, risking erroneous and dangerous decisions.	Implement human review and audit logging for high-stakes decisions supported by the LLM.
CRITICAL	System Prompt Leakage of Internal Secrets	`SYSTEM_PROMPT_WITH_SECRETS variable and chat_internal() function (lines ~117-136)`	The system prompt contains sensitive internal information such as database credentials, admin passwords, and internal URLs. The prompt instructs the LLM to hide its existence, but it can be easily probed and secrets leaked to users.	Never embed sensitive secrets in system prompts. Store secrets securely and design prompts without exposing confidential info.
HIGH	Vector Embedding Ingestion Without Validation and Namespace Isolation	`Function store_user_embedding (lines ~138-150)`	User content is embedded and stored in a shared vector database without validating content or isolating users' embeddings namespaces. This allows adversaries to poison or manipulate similarity searches and exfiltrate others' data.	Validate input content, perform adversarial checks on embeddings, and apply per-user namespace isolation in vector stores.
MEDIUM	Embedding API Calls Without Rate Limiting or Input Validation (Denial of Service)	`Function get_embedding (lines ~152-161)`	Embedding requests pass user inputs directly with no limit on input length or rate limiting, allowing attackers to generate large or numerous requests and cause denial-of-service.	Enforce input size limits and rate limiting for embedding API calls to prevent abuse.
HIGH	Retrieval-Augmented Generation (RAG) Query Without Access Control	`Function rag_query_no_access_control (lines ~163-179)`	The function retrieves documents from the vector store with no per-user access control. Users can retrieve and learn sensitive information belonging to other users.	Implement strict document-level access controls and namespace isolation for vector database queries.